```
# Numerical operations
import numpy as np

# Data handling (optional but useful)
import pandas as pd

# Install gensim if not already installed
!pip install gensim

# Pre-trained word embeddings
import gensim.downloader as api

# Dimensionality reduction
from sklearn.manifold import TSNE

# Visualization
import matplotlib.pyplot as plt
```

```
 /usr/local/lib/python3.12/dist-packages (4.4.0)
18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
n>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
/usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
```

```
# Load pre-trained GloVe embeddings (100 dimensions)
model = api.load("glove-wiki-gigaword-100")

# Print vocabulary size
print("Vocabulary size:", len(model.key_to_index))

# Display example vector
word = "king"
print(f"\nVector for '{word}':\n")
print(model[word])
print("\nVector shape:", model[word].shape)
```

```
[==================================================] 100.0% 128.1/128.1MB downlo
Vocabulary size: 400000

Vector for 'king':

[-0.32307  -0.87616   0.21977   0.25268   0.22976   0.7388   -0.37954
 -0.35307  -0.84369  -1.1113   -0.30266   0.33178  -0.25113   0.30448
 -0.077491 -0.89815   0.092496 -1.1407   -0.58324   0.66869  -0.23122
 -0.95855   0.28262  -0.078848  0.75315   0.26584   0.3422   -0.33949
  0.95608   0.065641  0.45747   0.39835   0.57965   0.39267  -0.21851
  0.58795  -0.55999   0.63368  -0.043983 -0.68731  -0.37841   0.38026
  0.61641  -0.88269  -0.12346  -0.37928  -0.38318   0.23868   0.6685
 -0.43321  -0.11065   0.081723  1.1569    0.78958  -0.21223  -2.3211
 -0.67806   0.44561   0.65707   0.1045    0.46217   0.19912   0.25802
```

```
  0.057194  0.53443  -0.43133  -0.34311   0.59789  -0.58417   0.068995
  0.23944  -0.85181   0.30379  -0.34177  -0.25746  -0.031101 -0.16285
  0.45169  -0.91627   0.64521   0.73281  -0.22752   0.30226   0.044801
 -0.83741   0.55006  -0.52506  -1.7357    0.4751   -0.70487   0.056939
 -0.7132    0.089623  0.41394  -1.3363   -0.61915  -0.33089  -0.52881
  0.16483  -0.98878 ]

Vector shape: (100,)
```

```
# Word categories
animals = ["dog", "cat", "lion", "tiger", "wolf", "elephant", "horse", "monkey"

cities = ["paris", "london", "india", "tokyo", "delhi", "mumbai", "brazil", "bh

technology = ["computer", "laptop", "internet", "software", "hardware", "keyboa
              "screen", "mobile", "robot"]

# Combine all words
words = animals + cities + technology

print("Total selected words:", len(words))
```

```
Total selected words: 30
```

```
# Extract word vectors
word_vectors = []

valid_words = []

for word in words:
    if word in model:
        word_vectors.append(model[word])
        valid_words.append(word)

word_vectors = np.array(word_vectors)

print("Shape of word vectors:", word_vectors.shape)
```

```
Shape of word vectors: (30, 100)
```

```
# Initialize t-SNE
tsne = TSNE(n_components=2, random_state=42, perplexity=5)

# Apply t-SNE
reduced_vectors = tsne.fit_transform(word_vectors)

print("Shape after reduction:", reduced_vectors.shape)
```

```
Shape after reduction: (30, 2)
```

```
plt.figure(figsize=(12, 8))

# Scatter plot
x = reduced_vectors[:, 0]
y = reduced_vectors[:, 1]

plt.scatter(x, y)

# Annotate words
for i, word in enumerate(valid_words):
    plt.annotate(word, (x[i], y[i]))

plt.title("t-SNE Visualization of Word Embeddings")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.grid(True)
plt.show()
```